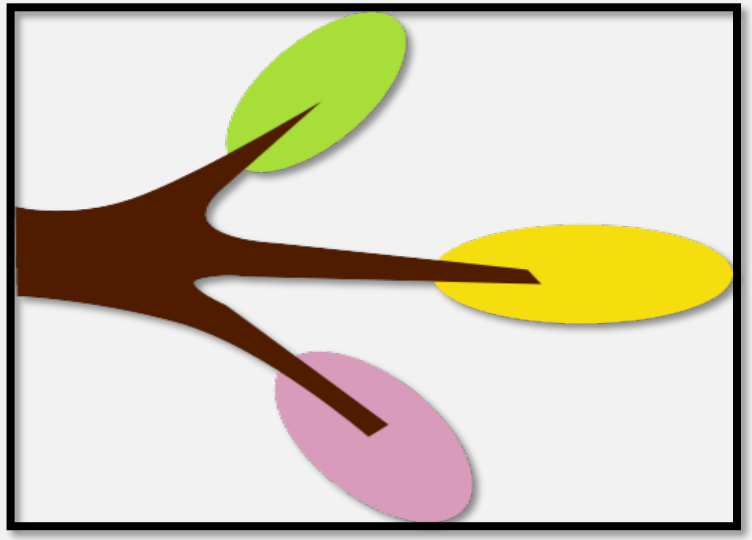
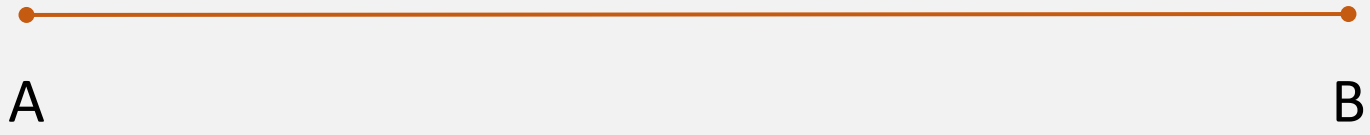
Conditional Statements

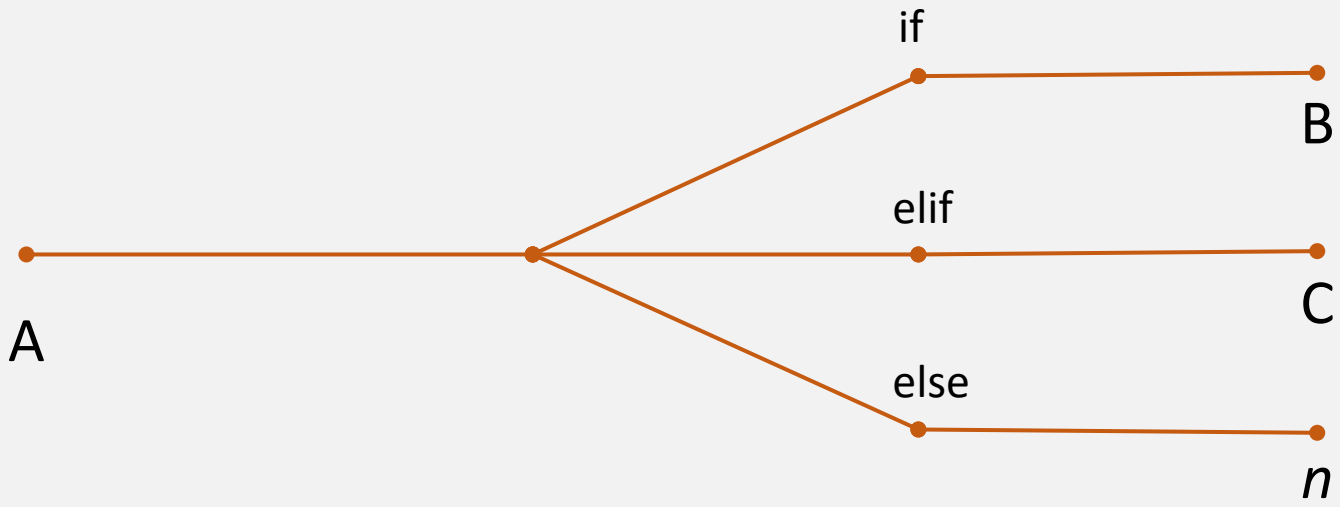
Branching Out



Current Programming Approach

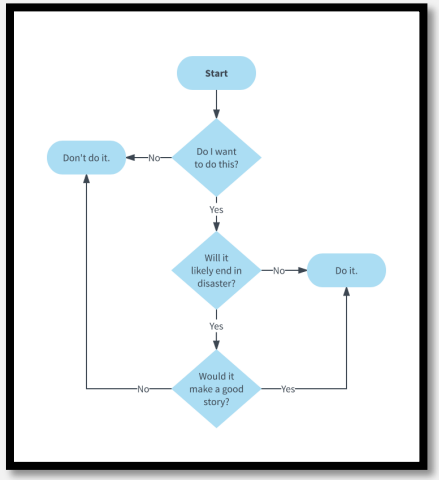


Conditionals Change This

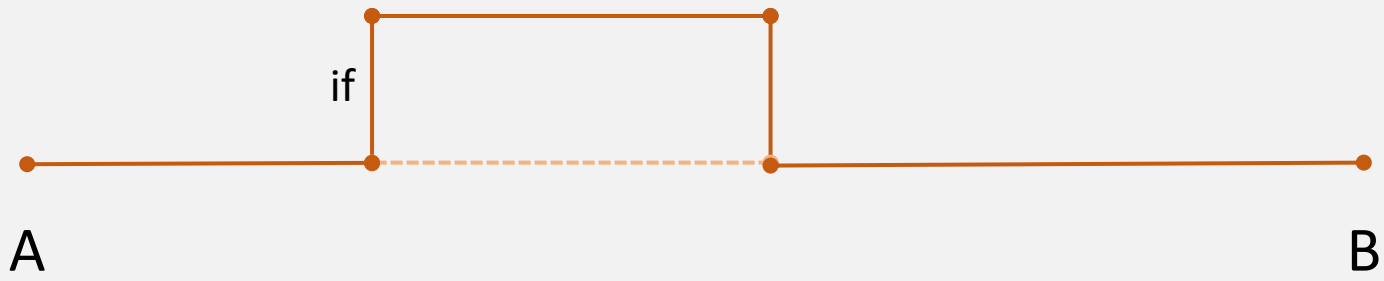


The if – elif – else Structure

Conditionals



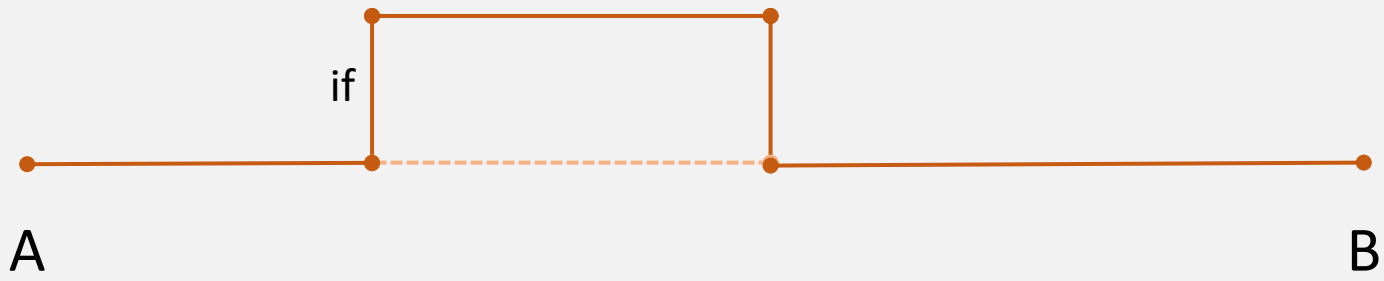
if



Runs only if a condition is met.

If the condition is not met, the code skips the if statement and continues running.

if




```
people = 100
chairs = 50

if people > chairs:
    print(”We need more chairs!”)
```

This indentation is necessary for the code to run properly.

if

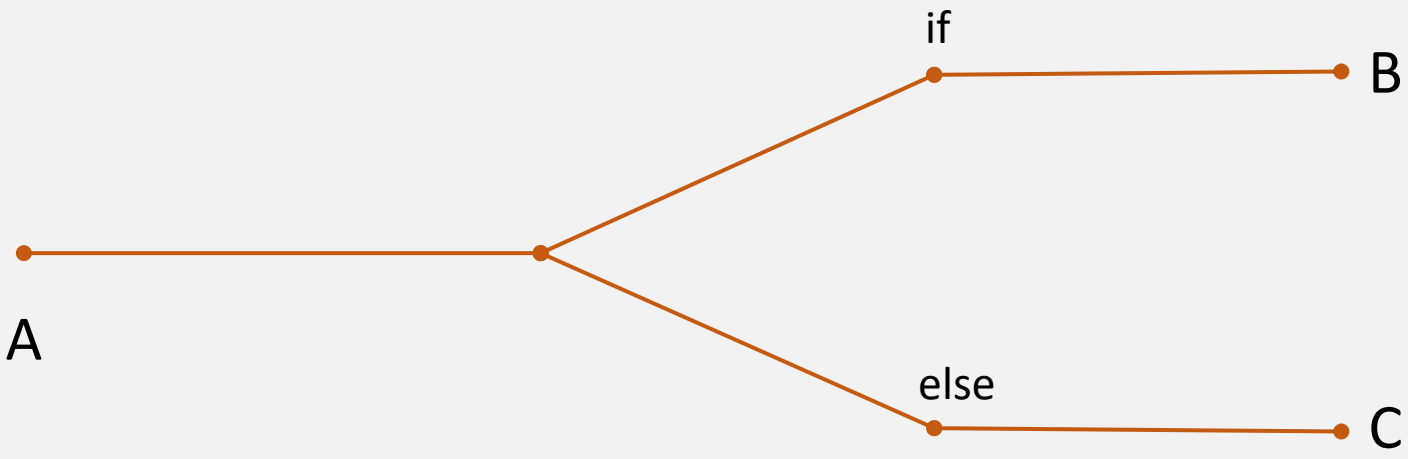
```
people = 100
chairs = 50

if people > chairs:
    print("We need more chairs!")
```

Condition is met.
The print statement will run.

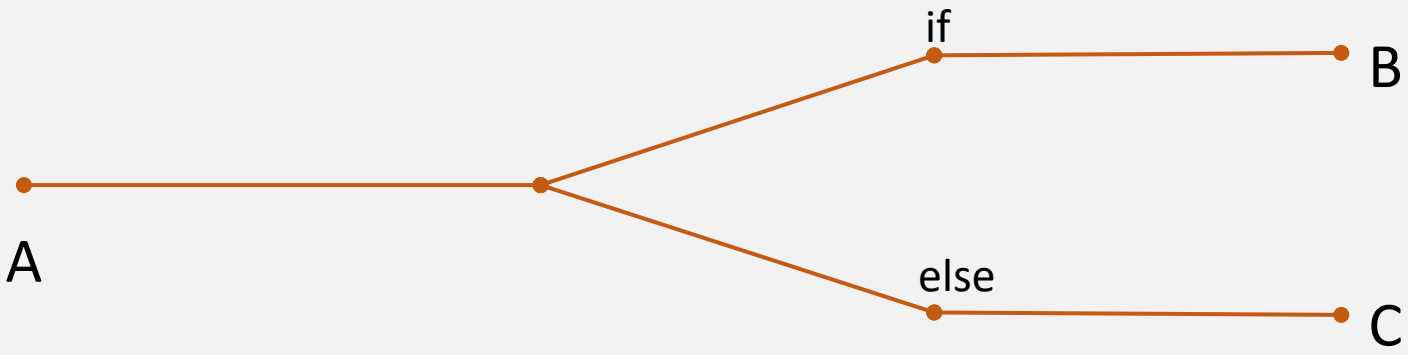
```
people = 100
chairs = 101

if people > chairs:
    print("We need more chairs!")
```

Condition is **not** met.
The print statement will **not** run.

if - else



if - else




```
people = 100
chairs = 50

if people > chairs:
       print(”We need more chairs!”)

else:
       print(“We have enough chairs.”)
```

if - else

```
people = 100
chairs = 50

if people > chairs:
        print(”We need more chairs!”)

else:
        print(“We have enough chairs.”)
```

Condition is met.
The **if** print statement will run.

if - else

```
people = 50
chairs = 100

if people > chairs:
        print(”We need more chairs!”)

else:
        print(“We have enough chairs.”)
```

Condition **not** is met.
The **else** print statement will run.

if - else

```
goldfish = input(prompt = “Do you like goldfish? (Y/N)”)

> Y

if goldfish == ‘Y’:
        print(“That’s like the dirtiest fish!”)

else:
        print(“Good. Did you know that goldfish tend to be very dirty?”)
```

```
That’s like the dirtiest fish!
```

if - else

```
cheese = input(prompt = """
If you're eating a cheese that isn't yours, what
kind of cheese is it?
""")

> Nacho Cheese


if cheese == 'Nacho Cheese':
        print("That's a great joke!")

else:
        print("Nacho cheese!")
```

```
That's a great joke!
```

if - elif - else

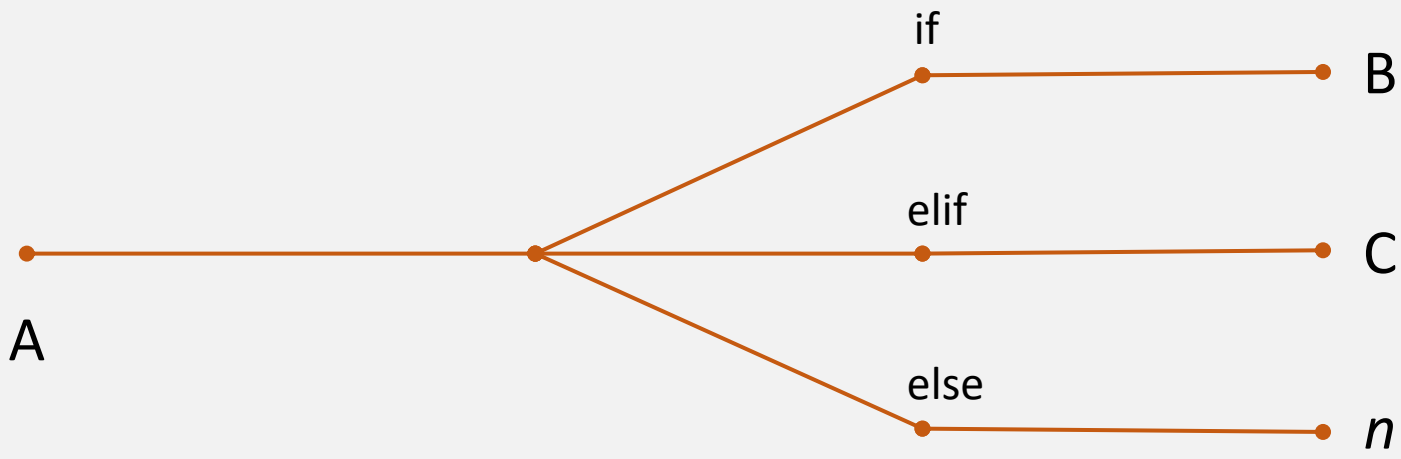
In Excel…

```
=IF(condition, output if true, output if false)
```

```
=IF(people > chairs, ”We need more chairs”,
IF(people == chairs, “Perfect! Good job
everyone!”, ”We have enough chairs.”), ”We
have enough chairs.”)
```

elif is an elegant solution for nested IF statements.

if - elif - else



if - elif - else

```
people = 100
chairs = 50

if people > chairs:
      print(”We need more chairs!”)


elif people == chairs:
      print(“Perfect! Great job everyone!”)


else:
      print(“We have enough chairs.”)
```

Condition is met.
The **if** print statement will run.

if - elif - else

```
people = 100
chairs = 100

if people > chairs:
        print(”We need more chairs!”)


elif people == chairs:
        print(“Perfect! Great job everyone!”)


else:
        print(“We have enough chairs.”)
```

Condition is met.
The **elif** print statement will run.

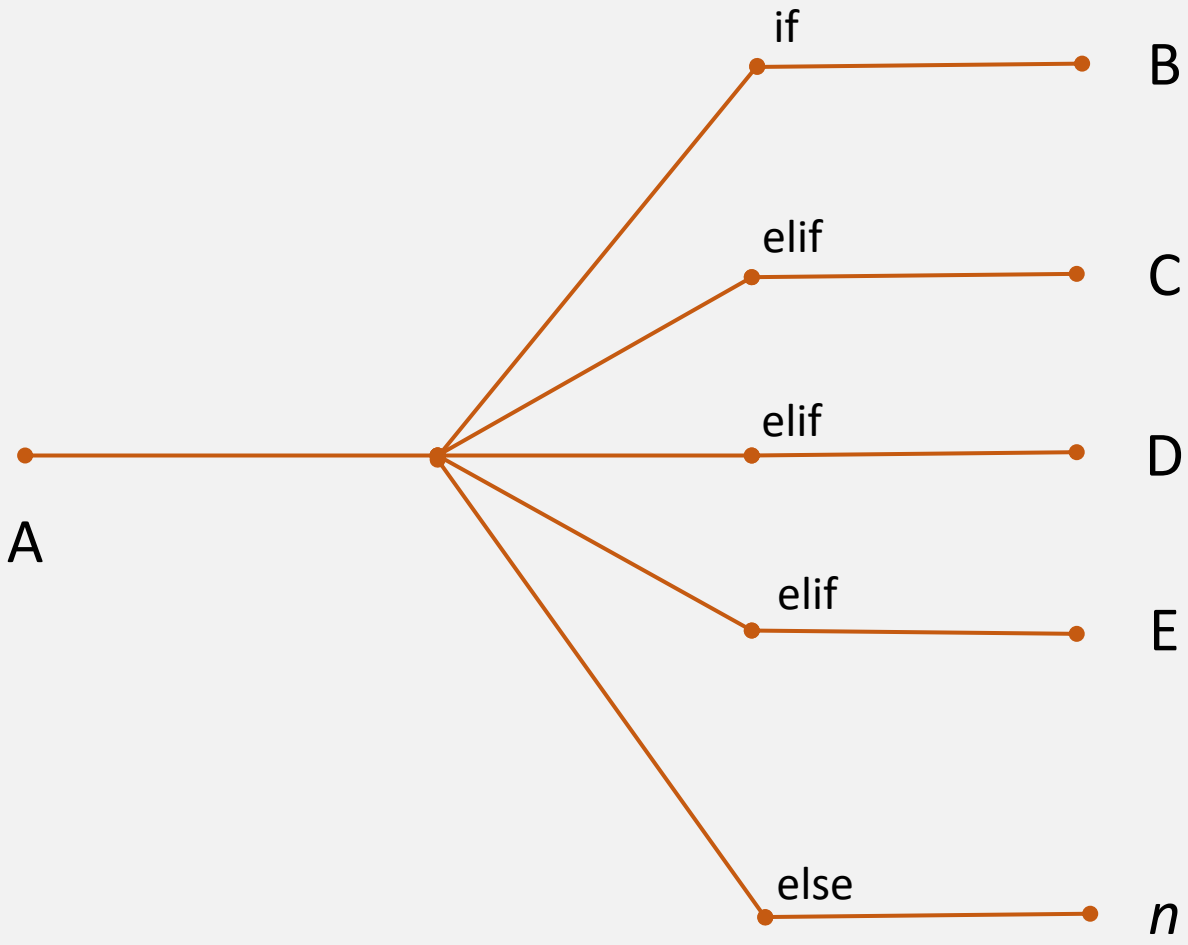
if - elif - else

```
people = 50
chairs = 100

if people > chairs:
        print(”We need more chairs!”)


elif people == chairs:
        print(“Perfect! Great job everyone!”)


else:
        print(“We have enough chairs.”)
```

No condition is met.
The **else** print statement will run.

if - elif - else

This can be extended as many times as needed.



if - elif - else

If two separate conditions are met, only the first one will execute.

```
people = 100

if people > 5:
        print("Some people are coming.")


elif people > 50:
        print("Many people are coming.")


else:
        print("Too many or too few people are coming.")
```

Both conditions are met
The **first** condition's print statement will run.

if - elif - else

```
answer = 5

guess  = input(prompt = “Guess a number between 1 and 10:  ”)


if guess == answer:
       print(“Great job!”)


elif guess != answer:
       print(“Sorry, that’s incorrect.”)


else:
       print(“Something went wrong, please try again.”)
```

if - elif - else

```
answer = 5

guess  = input(prompt = “Guess a number between 1 and 10:  ”)


if guess == answer:
      print(”Great job!”)


elif guess != answer:
      print(“Sorry, that’s incorrect.”)


else:
      print(“Something went wrong, please try again.”)
```

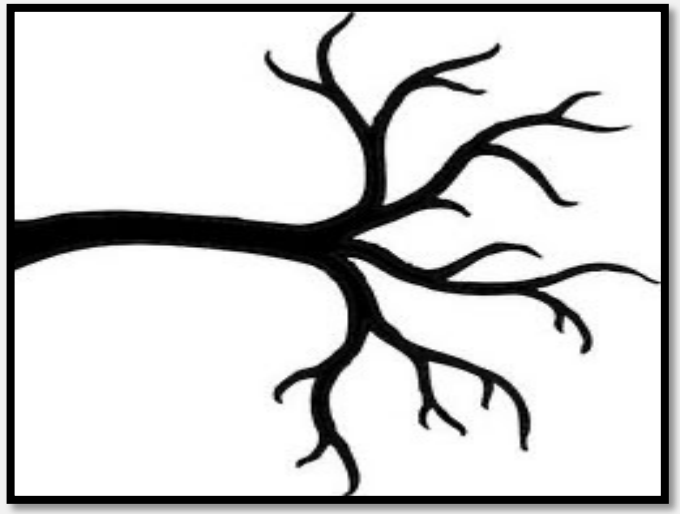
In this case, else will ALWAYS run.
Why?

if - elif - else

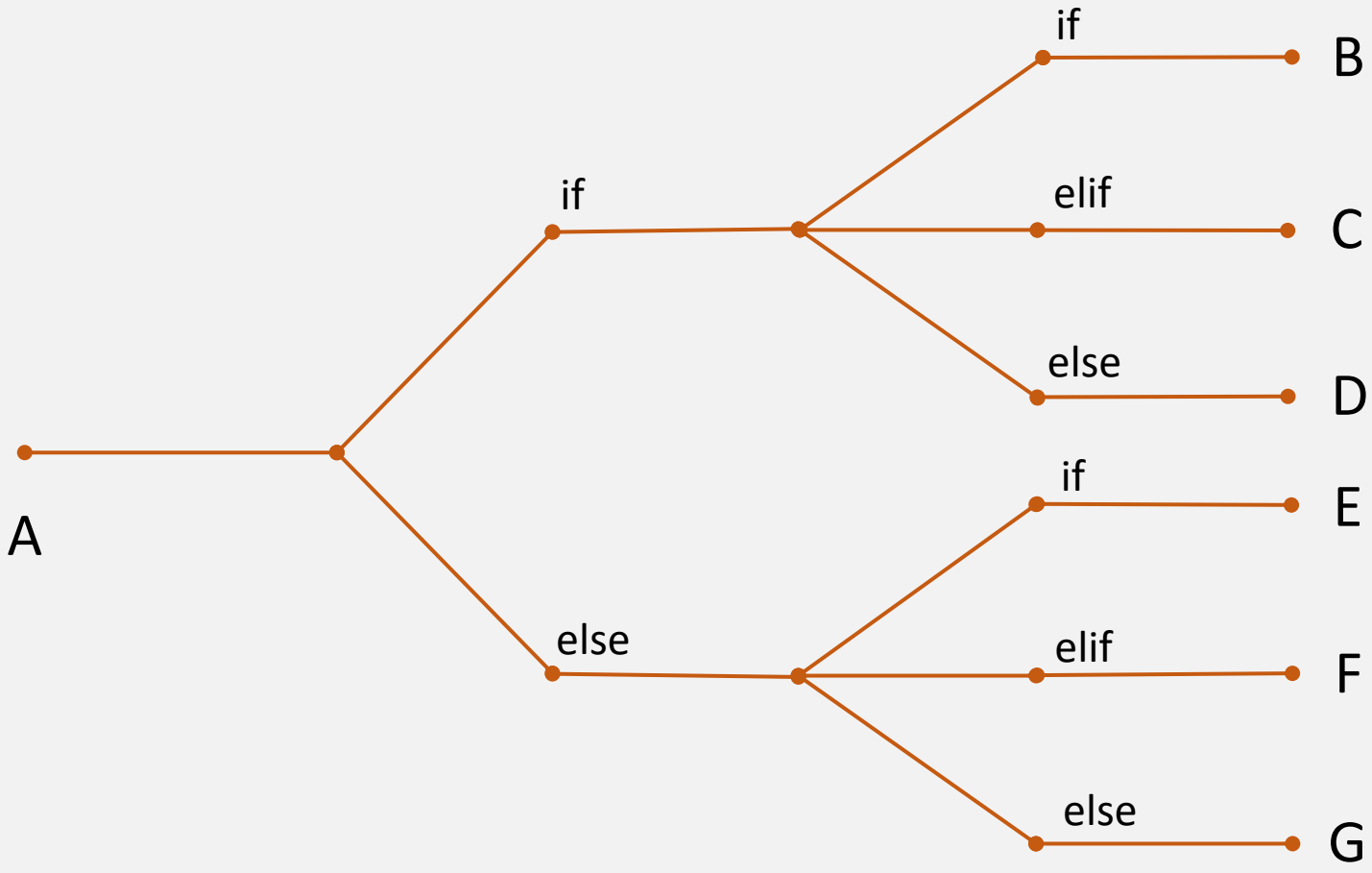
```
answer = 5

guess  = input(prompt = “Guess a number between 1 and 10:  ”)

print(type(answer))

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

 < class ‘int’>

 print(type(guess))

- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

< class ‘str’>
```

Nested Conditionals

Branching Further Out



Nested Conditionals



Nested Conditionals

These can get complicated very quickly.

A good practice is to build them up step-by-step.

Nested Conditionals – Starting Simple

```
people = 100
chairs = 50

if people > chairs:
       print(”We need more chairs!”)


elif people == chairs:
       print(“Perfect! Great job everyone!”)


else:
       print(“We have enough chairs.”)
```

Nested Conditionals – Building Up

```
people = 100
chairs = 50

if people > chairs:

        if chairs >= (0.90 * people):
                print("Maybe some people can stand.")

        else:
                print("We need more chairs!")

elif people == chairs:
        print("Perfect! Great job everyone!")

else:
        print("We have enough chairs.")
```

Nested Conditionals – Building Up

```
people = 100
chairs = 50

if people > chairs:

    if chairs >= (0.90 * people):
        print("Maybe some people can stand.")

    else:
        print("We need more chairs!")

elif people == chairs:
    print("Perfect! Great job everyone!")

else:
    print("We have enough chairs.")
```

Notice the different levels of indenting. This is also necessary for this code to run properly.

Logical and Boolean Operators

Learning to Simplify

$$\frac{\cancel{1}}{\cancel{3}_{1}} \times \frac{\cancel{1}}{\cancel{5}_{1}} = \frac{1}{6}$$

Logic Table

Command	Description
and	checks if two or more conditions are all True
or	checks if one of many conditions is True
not	checks if a condition is not True
!= (not equal)	checks if two objects are not the same
== (equal)	checks if two objects are the same
>=	checks if an object greater than or equal to something
<=	checks if an object less than or equal to something
True	checks to see if a condition is True
False	checks to see if a condition is False

Example: and

checks if one of many conditions is True

```
# not using ‘and’
x = 10

if x > 5:
	if x < 15:
		print(“x is between 5 and 15”)

	else:
		print(“x is NOT between 5 and 15”)

else:
	print(“x is NOT between 5 and 15”)
```

Example: and

checks if two or more conditions are all True

```
# using ‘and’
x = 10

if x > 5 and x < 15:
        print(“x is between 5 and 15”)

else:
        print(“x is NOT between 5 and 15”)
```

Two full expressions must be written.

Example: and

checks if two or more conditions are all True

```
# using ‘and’
x = 10

if x > 5 and < 15:
        print(“x is between 3 and 15”)

else:
        print(“x is NOT between 3 and 15”)
```

Two expressions must be written.
THIS WILL PRODUCE AN ERROR

Example: and

checks if two or more conditions are all True

```
# using ‘and’
x = 10
y = 11

if x > 5 and y < 15:
        print(“Both conditions are met”)

else:
        print(“At least one condition is not met”)
```

We do not need to use the same variable in each expression.

Example: or

checks if at least one of many conditions is True

```
# not using ‘or’
x = ‘USA’

if   x == ’Canada’:
       print(“North America”)

elif x == ‘USA’:
       print(“North America”)

elif x == ‘Mexico’
       print(“North America”)

else:
       print(“Not North America.”)
```

Example: or

checks if at least one of many conditions is True

```
# using ‘or’
x = ‘USA’

if x == ‘Canada’ or x == ‘USA’ or x == ‘Mexico’:
        print(“North America”)

else:
        print(“Not North America.”)
```

Notice that we are able to use more than one or operator in the same line of code.

This suit is black...

Example: in and not

checks if a condition is not True

```
# using ‘not’
lst = [1, 2, 3, 4, 5]

if 0 not in lst:
        lst.insert(0, 0)
```

Example: in and not

checks if a condition is not True

```
# using ‘not’
inventory = [‘robe’, ‘spell book’, ‘key’]

if ‘key’ not in inventory:
       print(“The door is locked and requires a key.”)

else:
       print(“You’ve used the key to open the door.”)
```

Example: in and not

checks if a condition is not True

```
# using ‘in’
inventory = [‘robe’, ‘spell book’, ‘key’]

if ‘key’ in inventory:
       print(“You’ve used your key to open the door.”)

else:
       print(“The door is locked and requires a key.”)
```

adding in

and other string manipulations

adding in

in is a powerful keyword that can help make your code more user-friendly.

its job is to recognize a character or phrase in a string and return True or False

it could be also used to recognize if an item is in a list

if - else

```
name = ‘pier’

‘p’ in name
```

Will return True because there is a ‘p’ in ‘pier’

- -

```
name = ‘enzo’

‘t’ in name
```

Will return False because there is no ‘t’ in ‘enzo’

if - else

```
name = ‘yuepeng’

‘G’ in name
```

Will return False because there is no big ‘G’ in ‘yuepeng’

- -

```
name = ‘umberto’

‘bert’ in name
```

Will return True because there is an ‘bert’ in ‘umberto’